Authentication & Authorization Service - Take-Home Coding Exercise

Overview

Build a service that validates OAuth2 access tokens and enforces fine-grained permissions using a SQLite database. The service should determine whether authenticated users can perform specific actions on resources.

Problem Statement

Create a permission checking service that:

1. Validates OAuth2 access tokens from providers like Auth0, AWS Cognito, or Ory
2. Extracts user identity from the validated token
3. Checks user permissions against a SQLite database
4. Returns authorization decisions (ALLOW/DENY)

Database Schema

You'll work with a pre-populated SQLite database containing one table:

```sql
CREATE TABLE user_permissions (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    user_id TEXT NOT NULL,
    action TEXT NOT NULL,    -- 'read', 'write', 'delete'
    resource TEXT NOT NULL,  -- 'transactions', 'accounts',
'wallets', or specific IDs
    effect TEXT NOT NULL     -- 'allow' or 'deny'
);
```

Sample Data

```sql
INSERT INTO user_permissions VALUES
(1, 'user123', 'read', 'transactions', 'allow'),
(2, 'user123', 'write', 'transactions', 'allow'),
```

```
(3, 'user123', 'delete', 'transactions', 'deny'),
(4, 'user123', 'read', 'accounts', 'allow'),
(5, 'user456', 'read', 'wallets/*', 'allow'),
(6, 'user456', 'write', 'wallets/wallet-789', 'allow'),
(7, 'user456', 'read', 'wallets/wallet-789/transactions',
'allow'),
(8, 'user789', 'write', 'wallets/*/transactions/*', 'allow'),
(9, 'admin789', 'read', '*', 'allow'),
(10, 'admin789', 'write', '*', 'allow'),
(11, 'admin789', 'delete', '*', 'allow');
```

Requirements

Core Functionality

1. **Token Validation**: Verify OAuth2 JWT tokens and extract user identity
2. **Permission Resolution**: Query the database to determine user permissions
3. **Authorization Logic**: Implement decision logic with proper precedence rules
4. **API Interface**: Create endpoints to check permissions

API Specification

POST /authorize

Request Body:

JSON
```
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6IkpXVCJ9...",
  "method": "GET",
  "path": "/wallets/wallet-789/transactions"
}
```

Response:

JSON
```
{
  "decision": "ALLOW",
  "user_id": "user456",
```

```
  "reason": "User has read permission for
wallets/wallet-789/transactions",
  "matched_permissions": [
    {
      "action": "read",
      "resource": "wallets/wallet-789/transactions",
      "effect": "allow"
    }
  ]
}
```

HTTP Method to Action Mapping

- `GET` → `read`
- `POST`, `PUT`, `PATCH` → `write`
- `DELETE` → `delete`

Path to Resource Mapping

Your service needs to map API paths to resource identifiers for permission checking. The exact mapping approach is up to you, but it should handle nested resources appropriately. Some examples:

- `/transactions`
- `/transactions/txn-123`
- `/accounts/acc-456`
- `/wallets`
- `/wallets/wallet-789`
- `/wallets/wallet-789/transactions`
- `/wallets/wallet-789/transactions/txn-456`

Design your resource mapping to work effectively with the permission patterns in your database.

Permission Resolution Rules

Design and implement a permission resolution system that handles hierarchical resources effectively:

1. **Specificity**: More specific resource paths should take precedence over general ones
2. **Deny beats Allow**: If both allow and deny permissions exist for the same resource, deny wins
3. **Wildcard Support**: Handle wildcard patterns appropriately (e.g., `*`, `wallets/*`)

4. **Nested Resource Matching**: Determine how permissions for parent resources apply to nested resources
5. **Default Deny**: If no matching permissions are found, default to DENY

The exact implementation of these rules is part of the design challenge - consider edge cases and document your approach.

Technical Requirements

Must Have

- **Language**: Choose from Python, Node.js (Typescript), Go, or Kotlin
- **JWT Validation**: Properly validate OAuth2 tokens (signature, expiration, audience)
- **Database**: SQLite integration with proper error handling
- **Testing**: Unit tests covering key scenarios
- **Documentation**: Clear README with setup and usage instructions

OAuth2 Provider Setup

You can register a free account at auth0, AWS Cognito, or ory.sh and use them as the OAuth2 provider for this exercise.

Architecture Considerations

- Clean separation of concerns (token validation, permission checking, business logic)
- **Resource mapping strategy**: How you convert API paths to permission resource identifiers
- **Permission matching algorithm**: Efficient approach for handling wildcards and nested resources
- Proper error handling and logging
- Input validation and sanitization
- Consider caching strategies for performance

Deliverables

Code Repository

- Well-structured codebase with clear separation of concerns
- README with setup and run instructions
- Requirements/dependencies file

Documentation

Include in your README:

- Setup instructions
- How to run the service

- API examples with curl commands
- Design decisions and trade-offs
- Areas for improvement

Test Coverage

- Unit tests for core authorization logic
- Integration tests for API endpoints
- Test data examples

Bonus Points

- Rate limiting or other security features
- Performance optimizations
- Audit logging for permission decisions

Submission Guidelines

1. Create a Git repository with your solution
2. Include a comprehensive README with setup instructions
3. Provide a brief video demo (2-3 minutes) of your working solution
4. Share the repository link and any additional notes

Sample Test Scenarios

Your service should handle these nested resource cases correctly:

Shell

```
# Should ALLOW - user123 has read access to transactions
curl -X POST localhost:8080/authorize -d '{
  "access_token": "valid_token_for_user123",
  "method": "GET",
  "path": "/transactions"
}'

# Should DENY - user123 explicitly denied delete access
curl -X POST localhost:8080/authorize -d '{
  "access_token": "valid_token_for_user123",
  "method": "DELETE",
  "path": "/transactions/txn-456"
}'
```

```
# Should ALLOW - user456 has specific permission for wallet-789
transactions
curl -X POST localhost:8080/authorize -d '{
  "access_token": "valid_token_for_user456",
  "method": "GET",
  "path": "/wallets/wallet-789/transactions"
}'

# Should ALLOW - user789 can write to any transaction in any
wallet
curl -X POST localhost:8080/authorize -d '{
  "access_token": "valid_token_for_user789",
  "method": "POST",
  "path": "/wallets/wallet-456/transactions/txn-999"
}'
curl -X POST localhost:8080/authorize -d '{
  "access_token": "valid_token_for_user789",
  "method": "POST",
  "path": "/wallets/wallet-789/transactions"
}'

# Should DENY - user456 only has read access to wallet-789
transactions, not write
curl -X POST localhost:8080/authorize -d '{
  "access_token": "valid_token_for_user456",
  "method": "POST",
  "path": "/wallets/wallet-789/transactions"
}'

# Should ALLOW - admin789 has global wildcard permissions
curl -X POST localhost:8080/authorize -d '{
  "access_token": "valid_token_for_admin789",
  "method": "DELETE",
  "path": "/accounts/acc-123/settings"
}'

# Test wildcard inheritance behavior (implementation dependent)
curl -X POST localhost:8080/authorize -d '{
  "access_token": "valid_token_for_user456",
```

```
  "method": "GET",
  "path": "/wallets/wallet-999"
}'
```

Questions?

If you have any questions about the requirements or need clarification on business rules, please don't hesitate to reach out. We encourage asking questions as it demonstrates thoughtful analysis of the problem.

In addition, feel free to make reasonable assumptions about requirements not explicitly specified. Document your assumptions in the README.